

Exp:03

Write the Python Program for Water Jug Problem

Aim:

To write a python program for water jug problem

Objective:

To understand state-space search by finding a sequence of operations (fill, empty, pour) that measures a target quantity of water using two jugs of different capacities.

Algorithm:

Step1: Start with two empty jugs of different capacities, and a target quantity of water to measure.

Step2: Fill one jug to its maximum capacity.

Step3: Pour the water from the full jug into the other jug until it is full or the first jug is empty.

Step4: If the second jug is full, pour out the water to empty it.

Step5: Repeat steps 2-4, filling and pouring water between the two jugs, until the desired quantity of water is measured or all possible combinations are tried.

Program:

```
from collections import deque

def water_jug():
    visited = set()
    queue = deque()

    start = (0, 0)
    goal = 2

    queue.append((start, [start]))
    visited.add(start)

    while queue:
        (a, b), path = queue.popleft()
```

```

if a == goal or b == goal:
    return path

next_states = [
    (4, b),          # Fill 4L jug
    (a, 3),          # Fill 3L jug
    (0, b),          # Empty 4L jug
    (a, 0),          # Empty 3L jug
    (max(0, a-(3-b)), min(3, b+a)), # Pour 4L -> 3L
    (min(4, a+b), max(0, b-(4-a))) # Pour 3L -> 4L
]

for state in next_states:
    if state not in visited:
        visited.add(state)
        queue.append((state, path + [state]))

path = water_jug()

print("Solution:")
for state in path:
    print(state)    if col >= N:
        return True

for row in range(N):
    if is_safe(row, col):
        board[row][col] = 1

    if solve(col + 1):
        return True

    board[row][col] = 0

```

```
    return False

if solve(0):
    print("Solution:")
    for row in board:
        print(*row)
else:
    print("No solution exists")
```

Output:

```
(0, 0)
(0, 3)
(3, 0)
(3, 3)
(4, 2)
```

Result: Hence, the simple python program for water jug problem is done